BUTTERFLYFX ANPC DESIGN SUITE

Manifold GLB Optimization Guide

Store Geometry Not Bytes — High-Fidelity 3D Assets Through Dimensional Programming

Document ID:	BFXG-006
Suite:	ButterflyFX ANPC Design Suite, Document #6
Author:	Kenneth Bingham
Portal:	kensgames.com
Engine:	ButterflyFX™
Version:	1.0
Date:	April 15, 2026

Table of Contents

1. Executive Summary
2. The Two Worlds: Traditional vs. Manifold GLB
3. Manifold Mathematics for Geometry
4. The ManifoldDescriptor Format
5. Traditional GLB Optimization (When You Must Store Meshes)
6. Hybrid GLB Architecture
7. GPU Tessellation Pipeline
8. Procedural Textures from Manifolds
9. Implementation Cookbook
10. Quality Assurance

11. Developer Workflow
12. Glossary
13. ButterflyFX Design Suite Index
14. References

1. Executive Summary

> "Store Geometry Not Bytes."
>
> — Kenneth Bingham, ButterflyFX Engine Architecture

1.1 The Problem

GLB files — the binary container format for glTF 3D assets — store every vertex position, normal vector, UV coordinate, and index as raw binary data. For a browser-based game engine like ButterflyFX, this creates a fundamental delivery problem:

- A detailed starfighter model with smooth hull panels, weapon hardpoints, and engine geometry can range from **15–50 MB** as a standard GLB.
- A full scene containing a space station, asteroid field, and multiple ships can exceed **200 MB**.
- Browser-based games hosted on **kensgames.com** must load over the network. Users on mobile connections cannot wait for hundreds of megabytes.
- GPU memory is finite — particularly on integrated graphics and mobile devices targeted by the ButterflyFX platform.

The geometry is the bottleneck. A single curved hull panel that appears visually smooth requires thousands of triangles, each stored as explicit vertex data. The mathematical *information content* of that curve is trivial — but the *storage cost* is enormous.

1.2 The Traditional Solution

Industry-standard tools attack this problem through compression and decimation:

- **Draco compression** — entropy-encodes vertex and index data, achieving 5–8x compression ratios.
- **meshopt quantization** — reduces per-vertex precision (14-bit positions, 8-bit normals) and reorders data for GPU cache efficiency.
- **Texture compression** — KTX2 containers with Basis Universal encoding, or WebP fallback, reducing texture payloads by 4–6x.
- **Mesh simplification** — removing vertices and collapsing edges to reduce triangle counts.

These techniques reduce file size but impose quality costs. Vertices are removed, normals are approximated, textures undergo lossy compression. At high compression ratios, surfaces develop faceting artifacts, UV seams widen, and silhouette quality degrades. The developer is perpetually negotiating between file size and visual fidelity.

1.3 The ButterflyFX Solution: Manifold-Native Geometry

ButterflyFX introduces a fundamentally different approach: **manifold-native geometry**. Instead of storing 50,000 vertices to describe a curved hull panel, store the **manifold function parameters** — a few bytes — and tessellate the surface at runtime on the GPU.

The foundational surfaces discovered by Kenneth Bingham in Blender 3D are:

- **$z = xy$** — the hyperbolic paraboloid (saddle surface)
- **$z = xy^2$** — the asymmetric parametric surface

Through composition, transformation, and deformation of these base surfaces, the ButterflyFX engine generates the full range of smooth mechanical and architectural geometry required by its game titles: starfighter hulls, track surfaces, Schwartz Diamond lattices, station modules, and procedural environments.

Key Insight

The geometry is *mathematically perfect* at every level of detail. There is no decimation — only tessellation density. A distant object uses 8 subdivisions; a close-up uses 64. The surface is identical in both cases. No LOD popping, no quality loss, no compression artifacts. The phrase **"Store Geometry Not Bytes"** captures this paradigm: the geometry is the function itself, not its sampled output.

2. The Two Worlds: Traditional vs. Manifold GLB

2.1 Traditional GLB Pipeline

The conventional asset pipeline for browser-based 3D games follows a well-established sequence:

15. **Author** — Model in Blender, Maya, or 3ds Max with subdivision surfaces and high-resolution textures.
16. **Export** — Export as .glb (binary glTF) containing mesh data, materials, textures, and animations.
17. **Optimize** — Run `gltfpack` for vertex cache optimization, quantization, and mesh merging.
18. **Compress** — Apply Draco or meshopt compression to geometry; KTX2/Basis Universal to textures.
19. **Deploy** — Serve compressed .glb files via CDN with gzip/brotli transport compression.

Attribute	Traditional GLB Pipeline
Compatibility	Universal — any glTF-compliant viewer, engine, or tool
Tooling	Mature ecosystem: Blender, gltfpack, glTF-Transform, Draco, Basis Universal
Predictability	WYSIWYG — the mesh you see in Blender is the mesh the user sees
LOD Strategy	Fixed — must author 3–4 LOD meshes manually or via simplification
File Size Scaling	Linear with geometric complexity — more detail = more bytes
Quality at Compression	Lossy — quantization introduces precision errors; simplification removes features
Runtime Adaptation	None — geometry is fixed at export time

2.2 Manifold-Native GLB Pipeline

The manifold pipeline replaces stored geometry with generative parameters:

20. **Author** — Model in Blender; identify surfaces that conform to manifold functions (saddle patches, tubes, swept profiles).
21. **Extract** — Derive manifold parameters: surface type, domain ranges, transform matrices, deformation functions.
22. **Encode** — Store parameters as **ManifoldDescriptor** JSON payloads (typically 64–512 bytes per patch).
23. **Runtime** — GPU or CPU tessellates surfaces from parameters at load time, generating BufferGeometry.
24. **Adapt** — Tessellation density adjusts per frame based on camera distance, screen-space error, and performance budget.

Attribute	Manifold-Native Pipeline
Storage Cost	Near-zero for smooth surfaces —

Attribute	Manifold-Native Pipeline
	function parameters only
LOD Strategy	Continuous — adjust tessellation density from 8 to 128+ segments in real time
Quality	Mathematically perfect at every LOD — no decimation artifacts
Runtime Adaptation	Full — screen-space error metric drives tessellation per patch per frame
Compatibility	Requires ButterflyFX shader pipeline; falls back to traditional mesh in standard viewers
Geometry Types	Best for smooth, parametric, mechanical, and architectural surfaces
Development Cost	Higher initial investment in tessellation pipeline and tooling

2.3 Hybrid Pipeline (Recommended)

The ButterflyFX production pipeline is a **hybrid approach** — using manifold generation where it excels and traditional GLB where it is more appropriate.

Use Manifold Generation For:

- Hull panels, wing surfaces, body panels (starfighters, vehicles)
- Architectural surfaces — station modules, hangars, corridors
- Track surfaces — banked curves, elevation changes, barriers
- Lattice structures — Schwartz Diamond unit cells and instances
- Procedural environments — asteroid surfaces, terrain patches, debris fields
- Shield bubbles, engine thrust cones, force-field geometry

Use Traditional GLB For:

- Character models and pilot figures
- Cockpit instruments, buttons, screens, HUD elements

- Organic objects — plants, cloth, biological entities
- Imported third-party assets without manifold decomposition
- Highly irregular geometry that resists parametric description

Hybrid GLB Format

A hybrid .glb file is a *fully standard glTF binary* that contains both traditional mesh data AND ManifoldDescriptor payloads in a custom extension (`BFXTM_manifold_geometry`). Any glTF viewer can open it and render the traditional meshes. The ButterflyFX loader detects the extension and substitutes manifold-generated geometry for tagged meshes, achieving dramatic size and quality improvements.

Pipeline Comparison Summary

Factor	Traditional	Manifold-Native	Hybrid (Recommended)
File size	Large (10–200 MB)	Minimal (<1 MB)	Small (1–6 MB)
Quality	Degrades with compression	Perfect at all LODs	Best of both
Compatibility	Universal	ButterflyFX only	Universal with BFX enhancements
LOD	Fixed, discrete	Continuous, adaptive	Continuous for manifold; fixed for traditional
Development cost	Low	High	Moderate
Coverage	All geometry types	Smooth/ parametric only	All geometry types

3. Manifold Mathematics for Geometry

This section defines the mathematical foundation upon which ButterflyFX's manifold geometry system is built. The two foundational surfaces — **z = xy** and **z = xy²** — are combined through composition, transformation, and deformation to produce the full range of smooth surfaces used across all ButterflyFX titles.

3.1 z = xy — The Saddle Surface

Mathematical Definition

The **hyperbolic paraboloid** (saddle surface) is defined by the function:

```
z = f(x, y) = x · y
```

Over a rectangular parameter domain $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$, this produces a surface with **positive curvature** along one diagonal and **negative curvature** along the other. Every cross-section parallel to the xz-plane or yz-plane is a straight line — making the surface a **doubly ruled surface** — while diagonal cross-sections are parabolas.

Parameter Space and UV Mapping

The natural **parametric UV mapping** maps directly from the (x, y) domain to (u, v) texture coordinates:

```
u = (x - x_min) / (x_max - x_min) v = (y - y_min) / (y_max - y_min)
```

This produces zero-distortion UV mapping for uniform texturing — no UV unwrapping required.

Geometry Applications

Application	Domain	Transforms Applied	ButterflyFX Title
Wing panels	[-1, 1] × [0, 3]	Scale, taper along y-axis	Starfighter
Hull curvature	[-0.5, 0.5] × [-0.5, 0.5]	Scale z by 0.2 (subtle curvature)	Starfighter

Application	Domain	Transforms Applied	ButterflyFX Title
Architectural arches	[-2, 2] × [-2, 2]	Rotate 45°, scale, translate	Starfighter (stations)
Track banking	[0, 1] × [0, track_length]	Bend along spline, scale z by bank_angle	Fast Track
Lattice junctions	[-1, 1] × [-1, 1]	Instanced at lattice nodes	Schwartz Diamond

Parameter Encoding

A single saddle patch requires the following stored data:

```
Surface type:     1 byte   ("xy") x_domain:         8 bytes  (two float32)
y_domain:         8 bytes  (two float32) Transform matrix: 48 bytes (mat4x3,
omitting last row [0,0,0,1])                    -------- Total:            ~65
bytes
```

Compare this to a tessellated mesh at 32 segments per axis: 32 × 32 = 1,024 vertices × 32 bytes each (position + normal + UV) = **32,768 bytes**. The manifold encoding is **500x smaller**.

Code Example: Three.js Saddle Patch

```
// Generate a saddle surface z = xy as Three.js BufferGeometry function
createSaddlePatch(xRange, yRange, segments, transform) {     const [xMin,
xMax] = xRange;     const [yMin, yMax] = yRange;     const positions = [];
const normals = [];     const uvs = [];     const indices = [];      for (let
j = 0; j <= segments; j++) {         for (let i = 0; i <= segments; i++) {
const u = i / segments;             const v = j / segments;             const
x = xMin + u * (xMax - xMin);             const y = yMin + v * (yMax - yMin);
const z = x * y;  // z = xy              positions.push(x, y, z);
uvs.push(u, v);              // Analytical normal: cross(dF/dx, dF/dy)
// dF/dx = (1, 0, y), dF/dy = (0, 1, x)             // normal = (-y, -x, 1)
normalized             const len = Math.sqrt(y*y + x*x + 1);
normals.push(-y/len, -x/len, 1/len);         }     }      for (let j = 0; j <
segments; j++) {         for (let i = 0; i < segments; i++)
{             const a = j * (segments + 1) + i;             const b = a + 1;
const c = a + (segments + 1);             const d = c + 1;
indices.push(a, c, b, b, c, d);         }     }      const geometry = new
THREE.BufferGeometry();     geometry.setAttribute('position',         new
THREE.Float32BufferAttribute(positions, 3));
geometry.setAttribute('normal',         new
THREE.Float32BufferAttribute(normals, 3));     geometry.setAttribute('uv',
new THREE.Float32BufferAttribute(uvs, 2));     geometry.setIndex(indices);
if (transform) {         geometry.applyMatrix4(transform);     }     return
geometry; }
```

3.2 $z = xy^2$ — The Asymmetric Surface

Mathematical Definition

```
z = f(x, y) = x · y²
```

This surface is **asymmetric** in its parameters: linear in x but quadratic in y. It produces geometry that curves gently in one direction while curving sharply in the other — ideal for tapered, intake-like shapes.

Analytical Normals

```
dF/dx = (1, 0, y²) dF/dy = (0, 1, 2xy) Normal = (-y², -2xy, 1) normalized
```

Geometry Applications

Application	Domain	Why $z = xy^2$	ButterflyFX Title
Engine nacelles	[-1, 1] × [0, 2]	Quadratic flare in y creates intake scoop shape	Starfighter
Intake scoops	[0, 1] × [-1, 1]	Parabolic cross-section with linear taper	Starfighter
Asymmetric fairings	[-0.5, 0.5] × [-1, 1]	One-sided curvature for aerodynamic profiles	Fast Track (car bodies)
Track elevation changes	[0, 1] × [0, L]	Accelerating curvature models natural terrain transitions	Fast Track
Lattice tube profiles	[-1, 1] × [-1, 1]	Asymmetric cross-section for Schwartz Diamond tubes	Schwartz Diamond

3.3 Surface Composition

Complex geometry is built by **composing multiple manifold patches** with continuity constraints at their boundaries.

Adjacency Stitching

When two patches share an edge, the tessellator ensures that both patches produce **identical vertex positions** along the shared boundary. This is achieved by:

- Constraining both patches to the same domain value along the shared edge
- Using identical tessellation segment counts along the shared edge
- Snapping boundary vertices to exact function evaluations (no floating-point rounding divergence)

Normal Continuity (C1)

For surfaces to appear smooth across patch boundaries, **C1 continuity** (matching surface tangents) is required. Adjacent patches achieve this through **Hermite interpolation** at boundaries: the surface position and first derivative are matched, producing seamless normal transitions.

Example: Starfighter Wing

A single starfighter wing is composed of **6 saddle patches**:

Patch	Surface	Role	Storage
1	z = xy	Upper wing root	64 bytes
2	z = xy	Upper wing tip (tapered)	64 bytes
3	z = xy	Lower wing root	64 bytes
4	z = xy	Lower wing tip (tapered)	64 bytes
5	z = xy	Leading edge blend	64 bytes
6	z = xy	Trailing edge blend	64 bytes
Total storage:	**384 bytes**		
Generated vertices (max tessellation):	**30,000+**		

3.4 Manifold Transforms

Every manifold patch is modified through a chain of **affine and non-affine transforms**:

Affine Transforms (mat4)

- **Translation** — position the patch in world space
- **Rotation** — orient the patch (quaternion or Euler)
- **Scale** — resize uniformly or along individual axes
- **Shear** — skew the patch for asymmetric profiles

Deformation Functions

- **Taper** — scale x and z progressively along y: `scale(y) = 1 - taper_rate * y` (2 parameters)
- **Twist** — rotate the cross-section progressively along an axis: `angle(y) = twist_rate * y` (1 parameter)
- **Bend** — curve the patch along a circular arc: `radius, axis, range` (4 parameters)

Each deformation adds only a few bytes to the ManifoldDescriptor, while producing complex organic-looking shapes that would require thousands of additional vertices in traditional storage.

Instance Arrays

One manifold definition combined with **N transform matrices** produces massive geometry from minimal data. This is the key to the Schwartz Diamond lattice: one unit cell definition (one ManifoldDescriptor) instanced 343 times (7 × 7 × 7 grid) via a compact array of translation matrices.

> **Instance Math**
>
> 343 instances × 48 bytes per transform = 16,464 bytes. Plus one ManifoldDescriptor at ~200 bytes = **~16.7 KB total** for a structure that stores

as 28.6 MB in traditional mesh format. That is a **1,700x compression ratio** with zero quality loss.

4. The ManifoldDescriptor Format

The **ManifoldDescriptor** is a JSON object that fully defines a manifold surface patch. It is stored within a GLB file as part of the `BFXTM_manifold_geometry` extension payload.

4.1 Schema Definition

```
{   "ManifoldDescriptor": {     "id": "string",                   // Unique
patch identifier     "surface_type": "xy | xy2 | custom",  // Base surface
function     "params": {       "x_domain": [min, max],        // Parameter
range for x       "y_domain": [min, max],        // Parameter range for y
"coefficients": [number],      // For custom surfaces: polynomial coeffs
"transform": [                 // 4x4 matrix (column-major, 16 floats)
1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1       ],       "deformations": [         {
"type": "taper | twist | bend",           "axis": "x | y | z",
"params": {             "rate": number,          // Taper/twist rate
"radius": number,        // Bend radius             "range": [start,
end]    // Active range along axis           }         }       ]     },
"tessellation": {       "min_segments": number,        // LOD floor (e.g., 4)
"max_segments": number,        // LOD ceiling (e.g., 128)       "adaptive":
boolean,           // Screen-space adaptive tessellation       "crack_free":
boolean          // Watertight stitching with neighbors     },
"material_id": "string",         // Reference to glTF material index
"uv_mapping": "parametric | planar | cylindrical | custom",     "neighbors":
["patch_id"],       // Adjacent patch IDs for C1 stitching     "lod_bias":
number               // Per-patch quality priority (-1 to +1)   } }
```

4.2 Example: Starfighter Wing Panel

```
{   "id": "sf_wing_upper_root",   "surface_type": "xy",   "params":
{     "x_domain": [-1.0, 1.0],     "y_domain": [0.0, 3.0],
"coefficients": [],     "transform": [       0.8, 0, 0, 0,       0, 1, 0, 0,
0, 0, 0.15, 0,       2.5, 0, 0.3, 1     ],     "deformations": [       {
"type": "taper",         "axis": "y",         "params": {           "rate":
0.35,           "range": [0.0, 3.0]         }       }     ]   },
"tessellation": {     "min_segments": 8,     "max_segments": 64,
"adaptive": true,     "crack_free": true   },   "material_id":
"mat_hull_titanium",   "uv_mapping": "parametric",   "neighbors":
[     "sf_wing_upper_tip",     "sf_wing_leading_edge",
"sf_fuselage_upper"   ],   "lod_bias": 0.2 }
```

4.3 Example: Schwartz Diamond Lattice Unit Cell

```
{   "id": "sd_unit_cell",   "surface_type": "custom",   "params":
{     "x_domain": [-3.14159, 3.14159],     "y_domain": [-3.14159, 3.14159],
"coefficients": [1.0, 1.0, 1.0, -1.0, -1.0, -1.0],     "transform":
[       1, 0, 0, 0,       0, 1, 0, 0,       0, 0, 1, 0,       0, 0, 0,
1     ],     "deformations": []   },   "tessellation": {     "min_segments":
12,     "max_segments": 48,     "adaptive": true,     "crack_free": true   },
"material_id": "mat_lattice_chrome",   "uv_mapping": "parametric",
"neighbors": [],   "lod_bias": 0.0,   "instances": {     "count": 343,
"grid": [7, 7, 7],     "spacing": [6.2832, 6.2832, 6.2832],     "origin": [-
18.8496, -18.8496, -18.8496]   } }
```

The Schwartz Diamond Equation

The Schwartz Diamond minimal surface is defined implicitly by `sin(x)sin(y)sin(z) + sin(x)cos(y)cos(z) + cos(x)sin(y)cos(z) + cos(x)cos(y)sin(z) = 0`. ButterflyFX uses a custom surface evaluator with the coefficients field encoding the trigonometric combination weights. The unit cell is then instanced across the 7×7×7 grid to form the complete Connect Four game board.

5. Traditional GLB Optimization (When You Must Store Meshes)

For geometry that cannot be represented as manifold patches — cockpit interiors, pilot models, complex mechanical assemblies — the standard GLB optimization pipeline applies. This section provides the complete reference for ButterflyFX developers.

5.1 Mesh Optimization with gltfpack

The `gltfpack` tool (part of the meshoptimizer library) is the primary optimization tool for traditional GLB assets in the ButterflyFX pipeline.

Recommended Command-Line Configurations

Title	Command	Notes
Starfighter	`gltfpack -i input.glb -o output.glb -cc -tc -si 0.5 -sa`	Meshopt compression, KTX2 textures, 50% simplification, simplify aggressively
Fast Track	`gltfpack -i input.glb -o output.glb -cc -tc -si 0.7`	Higher fidelity for close-up racing views; 70% simplification ratio
Schwartz Diamond	`gltfpack -i input.glb -o output.glb -cc -tc -si 0.4`	Traditional meshes are only marbles/UI; aggressive simplification safe
Archival	`gltfpack -i input.glb -o output.glb -tc`	Texture compression only; no mesh simplification; full quality preserved

Key gltfpack Flags Reference

Flag	Purpose
`-c`	Apply meshopt compression (EXT_meshopt_compression)
`-cc`	Apply enhanced meshopt compression with better ratio
`-tc`	Compress textures to KTX2/Basis Universal (requires KHR_texture_basisu)
`-tw`	Compress textures to WebP (requires EXT_texture_webp)
`-si R`	Simplify meshes to ratio R (0.0–1.0); 0.5 = retain 50% of triangles
`-sa`	Simplify aggressively (allows topology changes)
`-te`	Embed textures into binary output

5.2 Geometry Compression

meshopt vs. Draco Comparison

Factor	meshopt (EXT_meshopt_compression)	Draco (KHR_draco_mesh_compression)
Decode speed	~1 GB/sec (WASM SIMD)	~100 MB/sec (WASM)
Compression ratio	3–5x (geometry only)	5–8x (geometry only)
Post-gzip improvement	Excellent — designed for secondary compression	Marginal — already entropy-coded
Quality	Quantization only — topology preserved	Can alter topology and vertex order
Three.js support	r122+ via GLTFLoader.setMeshoptDecoder()	r92+ via DRACOLoader
Animation support	Yes — compresses all buffer views	Geometry only — does not compress animations

ButterflyFX Recommendation

Use **meshopt** for all runtime assets (faster decode = faster load). Use **Draco** only for archival storage where decode latency is not a concern. Always apply gzip or brotli transport compression on top of meshopt — the combination achieves ratios close to Draco with 10x faster decode.

Quantization Bit Depths

Attribute	Recommended Bits	Precision	Byte Savings vs. float32
Position	14-bit (normalized int16)	~0.006% of bounding box	50%
Normal	8-bit (octahedral encoding)	~1.4° angular error	75%

Attribute	Recommended Bits	Precision	Byte Savings vs. float32
UV (texcoord)	12-bit (normalized int16)	Sub-pixel at 4K textures	50%
Color	8-bit per channel	Standard LDR precision	75%
Joint weights	8-bit (normalized uint8)	Adequate for skinning	75%

Three.js meshopt Decoder Setup

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js'; import
{ MeshoptDecoder } from 'three/addons/libs/meshopt_decoder.module.js';  const
loader = new GLTFLoader(); loader.setMeshoptDecoder(MeshoptDecoder);
loader.load('starfighter_cockpit.glb', (gltf) => {     scene.add(gltf.scene);
});
```

5.3 Texture Optimization

KTX2 with Basis Universal

KTX2 (Khronos Texture 2.0) containers with **Basis Universal** encoding provide GPU-native texture compression that transcodes at load time to the optimal format for the user's hardware (BC1/BC3 on desktop, ETC2 on mobile, ASTC on modern mobile).

Texture Type	Basis Mode	Compression Ratio	Quality Note
Base color (diffuse/albedo)	ETC1S	6–8x vs. PNG	Good; slight color banding possible
Normal maps	UASTC	3–4x vs. PNG	High; preserves surface detail critical for lighting
ORM (occlusion/roughness/metallic)	UASTC	3–4x vs. PNG	High; channel independence preserved
Emissive	ETC1S	6–8x vs. PNG	Good; emissive details are lower frequency

Resolution Tiers

Tier	Resolution	Use Case
Hero	4096 × 4096	Player ship, primary vehicle, close-up UI elements
Mid	2048 × 2048	Enemy ships, track-side objects, station modules
Background	1024 × 1024	Distant objects, environmental geometry
LOD	512 × 512	Distant LODs, particle textures, UI icons

Normal Map Optimization

- Use **octahedral encoding** for normal maps — encode the X and Y components in two channels; reconstruct Z in the shader as `z = sqrt(1 - x*x - y*y)`.
- Drop the blue channel entirely — saves 33% of normal map storage with zero quality loss.
- Apply UASTC encoding (not ETC1S) to preserve the per-pixel angular accuracy required for correct lighting.

5.4 Animation Optimization

- **Keyframe reduction** — Remove redundant keyframes where interpolation produces identical results. gltfpack performs this automatically via `-sa` (simplify animations) flag.
- **Resampling** — Resample animations at a fixed rate (e.g., 30 fps) to eliminate unnecessary keyframes from high-rate motion capture data.
- **Quantized animation tracks** — meshopt compresses animation buffer views alongside geometry, achieving 2–4x reduction on animation data.

- **Skeletal vs. morph target tradeoffs** — Skeletal animation stores per-bone transforms (compact); morph targets store per-vertex deltas (large). Prefer skeletal animation where possible. Reserve morph targets for facial expressions and soft-body deformation only.

6. Hybrid GLB Architecture

6.1 File Structure

A hybrid GLB file is a fully compliant glTF 2.0 binary that contains both traditional mesh data and manifold descriptors. The file anatomy:

```
+-------------------------------------------------------+ |  GLB Header (12
bytes)                                  | |    magic: 0x46546C67 ("glTF")
| |    version: 2                                         | |    length:
total file size                             |
+-------------------------------------------------------+ |  JSON Chunk
| |    Standard glTF JSON with:                           | |    - nodes,
meshes, materials, textures, animations   | |    - extensionsUsed:
["BFXTM_manifold_geometry",      | |        "KHR_mesh_quantization",
| |        "EXT_meshopt_compression",                     | |
"KHR_texture_basisu"]                           | |    - Per-mesh extension:
| |        "extensions": {                                | |
"BFXTM_manifold_geometry": {                 | |            "descriptors":
[ManifoldDescriptor, ...],  | |            "fallback_mesh": 0  // traditional
mesh idx| |          }                                           |
|        }                                                |
+-------------------------------------------------------+ |  Binary Chunk
| |    - Traditional mesh data (meshopt compressed)       | |    - Texture
data (KTX2/Basis Universal)              | |    - ManifoldDescriptor payloads
(JSON, <1 KB each)   | |    - Instance transform arrays (binary float32)
| +-------------------------------------------------------+
```

> **Graceful Fallback**
>
> The `fallback_mesh` field points to a traditional mesh stored in the same GLB. If the loader does not support `BFXTM_manifold_geometry`, it renders the fallback mesh instead. This means hybrid GLBs work in *any* glTF viewer — they just don't benefit from manifold quality and size advantages.

6.2 Loader Architecture

The ButterflyFX extended loader follows this sequence:

25. **Parse** — Standard GLTFLoader reads the JSON chunk and identifies extensions.
26. **Detect** — Check for `BFXTM_manifold_geometry` in each mesh node's extensions.
27. **Generate** — For each detected ManifoldDescriptor, invoke the **ManifoldGeometryGenerator** class to build a Three.js `BufferGeometry` from function parameters.
28. **Substitute** — Replace the fallback mesh's geometry with the manifold-generated geometry.
29. **LOD Register** — Register each manifold mesh with the LOD system for adaptive tessellation.
30. **Fallback** — If generation fails or extension is unknown, render the traditional mesh data.

ManifoldGeometryGenerator Class (Pseudocode)

```
class ManifoldGeometryGenerator {      generate(descriptor) {         const {
surface_type, params, tessellation } = descriptor;         const segments =
this.computeSegments(tessellation);          let evaluator;         switch
(surface_type) {             case 'xy':  evaluator = (x, y) => x * y; break;
case 'xy2': evaluator = (x, y) => x * y * y; break;             case
'custom': evaluator = this.buildCustom(params.coefficients);         }
const geometry = this.tessellate(             evaluator, params.x_domain,
params.y_domain, segments         );         this.applyDeformations(geometry,
params.deformations);         geometry.applyMatrix4(new
THREE.Matrix4().fromArray(params.transform));         return geometry;     }
tessellate(fn, xDomain, yDomain, segments) {         // Evaluate surface,
compute normals, generate indices         // (See Section 3.1 code example)
}      computeSegments(tessellation) {         if (tessellation.adaptive) {
return this.screenSpaceSegments(tessellation);         }         return
tessellation.max_segments;     } }
```

6.3 Size Comparison Table

The following table compares file sizes for key assets across all three ButterflyFX titles:

Asset	Traditional GLB	gltfpack Optimized	Manifold Hybrid	Reduction
Starfighter hull	12.4 MB	3.1 MB	0.8 MB	94%
Space station ring	45.2 MB	11.3 MB	2.1 MB	95%
Schwartz Diamond 7×7×7 lattice	28.6 MB	7.2 MB	0.04 MB	**99.8%**
Fast Track banked curve	8.3 MB	2.1 MB	0.3 MB	96%
Cockpit interior (traditional)	15.7 MB	3.9 MB	3.9 MB	75%
Full Starfighter scene	~85 MB	~21 MB	~4 MB	95%
Full Fast Track scene	~120 MB	~30 MB	~6 MB	95%

The Schwartz Diamond Showcase

The Schwartz Diamond lattice is the ultimate demonstration of manifold-native geometry. 343 unit cells, each defined by **one ManifoldDescriptor**, instanced 343 times via a translation matrix array = **~40 KB total** vs. 28.6 MB stored mesh. The lattice is mathematically perfect at every zoom level, with adaptive tessellation providing smooth close-ups of individual lattice nodes during marble-drop animations.

7. GPU Tessellation Pipeline

7.1 WebGL 2.0 Approach (Current)

WebGL 2.0 does not expose hardware tessellation stages (no tessellation control/evaluation shaders). ButterflyFX therefore performs tessellation in **JavaScript on the CPU**, generating `BufferGeometry` that is uploaded to the GPU as standard vertex/index buffers.

ManifoldTessellator Class

- **Surface evaluation** — evaluates the manifold function at each grid point, computing position, analytical normal, and UV coordinates.
- **Index generation** — generates triangle indices for the tessellated grid with consistent winding order.
- **Deformation application** — applies taper, twist, and bend deformations as post-processing on vertex positions and normals.
- **Transform application** — applies the ManifoldDescriptor's mat4 transform as the final step.

Web Workers for Background Tessellation

Off-screen manifold patches are tessellated in **Web Worker threads** to avoid blocking the main render loop. The workflow:

31. Main thread identifies which patches need tessellation changes (LOD transitions, new patches entering view).
32. Dispatches ManifoldDescriptor + desired segment count to a worker pool.
33. Worker evaluates surface, generates typed arrays (Float32Array for positions/normals/UVs, Uint32Array for indices).
34. Transfers arrays back to main thread via `postMessage` with `Transferable` buffers (zero-copy).
35. Main thread updates `BufferGeometry` attributes and uploads to GPU.

BufferGeometry Recycling Pool

To avoid garbage collection pressure from frequent LOD transitions, ButterflyFX maintains a **pool of pre-allocated BufferGeometry objects**. When a patch needs re-tessellation, an existing geometry is resized and reused rather than creating a new one.

7.2 WebGPU Approach (Future-Ready)

WebGPU exposes **compute shaders** that enable true GPU-side tessellation:

36. **Compute pass** — A compute shader evaluates the manifold function at each grid point, writing positions, normals, and UVs directly into a GPU storage buffer.
37. **Index generation** — A second compute pass generates the index buffer.
38. **Render pass** — The vertex shader reads directly from the computed buffers — no CPU-GPU transfer required.

This eliminates the CPU tessellation bottleneck entirely and enables:

- **Per-frame adaptive tessellation** — re-tessellate every patch every frame based on camera position, at GPU speeds.
- **Continuous LOD** — no discrete LOD switches, no popping, no visual discontinuities.
- **Compute-driven culling** — patches behind the camera or occluded by other geometry are never tessellated.

> **WebGPU Status**
>
> As of 2026, WebGPU is available in Chrome, Edge, and Firefox. ButterflyFX's WebGPU backend is in active development. The manifold tessellation pipeline will be the first compute workload ported to WebGPU, as it provides the highest return on investment.

7.3 LOD System

Traditional LOD vs. Manifold LOD

Factor	Traditional LOD	Manifold LOD
LOD count	3–4 pre-authored meshes	Infinite (continuous)
Transition	Hard switch — visible popping	Smooth — tessellation density changes gradually
Storage	Each LOD stored separately (3–4x mesh data)	One ManifoldDescriptor, all LODs derived at runtime
Quality	Varies by LOD level — lower LODs show simplification artifacts	Identical surface shape at all LODs
Authoring	Manual or automated simplification	Automatic — just set min/max segments

Screen-Space Error Metric

The manifold LOD system uses a **screen-space edge deviation metric**:

39. For each patch edge at the current tessellation level, compute the maximum deviation between the straight tessellated edge and the true manifold curve.
40. Project this deviation into screen space (pixels) using the current camera parameters.
41. If the deviation exceeds the **threshold** (default: 0.5 pixels), increase tessellation.
42. If the deviation is below the **lower threshold** (default: 0.1 pixels), decrease tessellation to save budget.

Per-Patch Priority (lod_bias)

The `lod_bias` field in ManifoldDescriptor assigns relative importance:

- **+1.0** — Maximum priority. Always tessellated at or near max_segments. Use for surfaces facing the camera or the player's cockpit view.

- **0.0** — Standard priority. Follows screen-space error metric normally.
- **-1.0** — Minimum priority. Tessellated at or near min_segments. Use for surfaces that are rarely in close-up view.

The LOD system distributes a **global tessellation budget** (default: 500,000 triangles per frame) across all visible manifold patches, weighted by their lod_bias and screen coverage.

8. Procedural Textures from Manifolds

8.1 The Zero-Texture Approach

For metallic, synthetic, and architectural surfaces — which constitute the majority of geometry in Starfighter, Fast Track, and Schwartz Diamond — the manifold surface itself contains enough mathematical information to **generate textures procedurally** without storing any texture images.

Curvature-Driven Material Properties

Manifold Property	Derived Texture	Logic
Surface curvature (Gaussian)	Roughness map	Concave regions = higher roughness (grease/dirt collects); convex regions = smoother (wear polishes)
Surface curvature (Mean)	Ambient occlusion	High-curvature crevices = darker AO; flat/convex regions = lighter
Normal variation rate	Edge wear/highlighting	Rapid normal changes = exposed edges = lighter metallic appearance
Parametric UV position	Panel lines, patterns	Regular intervals in UV space = panel seams, grip textures, warning stripes

Shader Implementation (Fragment Shader Pseudocode)

```
// In fragment shader: float curvature = computeGaussianCurvature(vPosition,
vNormal); float roughness = mix(0.15, 0.65, smoothstep(-0.5, 0.5,
curvature)); float ao = 1.0 - smoothstep(0.0, 2.0, abs(meanCurvature)) * 0.4;
// Panel lines from UV float panelLine = step(0.98, fract(vUV.x *
panelCount))                 + step(0.98, fract(vUV.y * panelCount));
roughness = mix(roughness, 0.8, panelLine * 0.5);  // Output PBR values
gl_FragColor = vec4(baseColor * ao, 1.0); // roughness and metallic written
to MRT or combined output
```

8.2 Hybrid Texturing

The zero-texture approach covers ~80% of surface area in a typical ButterflyFX scene. The remaining 20% uses small **detail textures** layered on top:

- **Decals** — squadron insignia, ship numbers, sponsor logos (Fast Track), hazard markings
- **Dirt and damage** — procedural noise-based grunge overlaid using triplanar mapping
- **Emissive details** — engine glow, cockpit screens, navigation lights

This hybrid texturing approach saves **80%+ of texture storage** for mechanical and architectural assets while maintaining visual richness.

9. Implementation Cookbook

This section provides title-specific guidance for applying manifold optimization across the ButterflyFX game portfolio.

9.1 Starfighter Assets

Manifold-Generated Components

Component	Surface Type	Patch Count	Storage
Hull panels (fuselage)	z = xy (saddle patches)	8	~512 bytes
Wings (left + right)	z = xy (tapered saddles)	6 per wing (12 total)	~768 bytes

Component	Surface Type	Patch Count	Storage
Engine nacelles	$z = xy^2$ (asymmetric tubes)	4 per nacelle (8 total)	~512 bytes
Shield bubble	$z = xy$ (rotated saddle = sphere approx.)	6 (cube-mapped patches)	~384 bytes
Tail fins	$z = xy$ (with taper deformation)	4	~256 bytes

Traditional GLB Components

- Cockpit interior — instruments, screens, controls, seat
- Pilot model — skeletal mesh with morph targets for visor animation
- Weapon hardpoints — turrets, missile racks (complex mechanical assemblies)
- Landing gear — articulated mechanical structure

Expected Scene Budget

Component	Traditional	Hybrid
Player ship	15 MB	1.2 MB
Enemy ships (x4)	40 MB	2.0 MB
Space station	45 MB	2.1 MB
Asteroids (x20)	10 MB	0.5 MB
Skybox + effects	2 MB	2.0 MB
Total	**~85 MB**	**~4 MB**

9.2 Schwartz Diamond Connect Four

The Manifold Lattice

The entire 7×7×7 Schwartz Diamond lattice is generated from **one ManifoldDescriptor** plus an instance matrix array:

Component	Approach	Storage
Lattice unit cell	1 ManifoldDescriptor (custom surface type)	~200 bytes
Instance transforms	343 translation matrices (7×7×7 grid)	~16.5 KB
Marbles (max 294)	Traditional sphere GLB, instanced	~2 KB (geometry) + instance buffer
Game board frame/base	4 manifold patches (xy saddles)	~256 bytes
UI elements	Traditional 2D sprites	~20 KB
Total game board:	**~50 KB**	

Marble Drop Animation

During the marble drop sequence, the camera zooms into the lattice structure. The adaptive LOD system increases tessellation on the visible lattice cells from 12 segments (distant view) to 48 segments (close-up), providing smooth, artifact-free geometry throughout the animation — all generated from the same ~200-byte ManifoldDescriptor.

9.3 Fast Track

Track Surface Generation

The racing track surface is generated from a combination of manifold primitives:

- **Track centerline** — defined as a 3D spline (Catmull-Rom) with control points
- **Track cross-section** — a $z = xy$ saddle patch defining banking angle relative to track width
- **Elevation changes** — $z = xy^2$ patches providing accelerating vertical curvature for hills and valleys
- **Composition** — the cross-section is swept along the spline, with banking and elevation parameters varying per segment

Asset Budget

Component	Approach	Traditional	Hybrid
Track surface	Manifold (spline + banking function)	35 MB	0.5 MB
Barriers and walls	Manifold panels, instanced	15 MB	0.2 MB
Grandstands	Manifold architectural panels	20 MB	0.8 MB
Pit buildings	Manifold (walls, roofs) + traditional (details)	12 MB	1.5 MB
Cars (body panels)	Manifold body + traditional wheels/details	25 MB	2.0 MB
Environment	Traditional (trees, signs, crowd)	13 MB	1.0 MB
Total track scene:	**~120 MB**	**~6 MB**	

10. Quality Assurance

10.1 Visual Quality Metrics

Metric	Method	Threshold
Surface deviation	Hausdorff distance between tessellated mesh and reference (high-res Blender export)	< 0.01 units at max tessellation
Normal angle deviation	Maximum angle between tessellated normal and analytical normal	< 2° at max tessellation
Silhouette smoothness	Edge detection on rendered silhouette — count visible faceting	0 visible faceting at target view distance

Metric	Method	Threshold
	edges	
UV distortion	Checker-pattern rendering — inspect for stretching or compression	< 5% area distortion across patch
Patch boundary continuity	Normal discontinuity visualization at patch edges	No visible seams in diffuse or specular lighting

10.2 Performance Metrics

Metric	Budget	Measurement Method
Tessellation budget	500,000 triangles max per frame	renderer.info.render.triangles (Three.js)
Load time	< 3 seconds for full scene	Performance.mark() from fetch start to first render
GPU memory	< 256 MB geometry buffer	WebGL memory extension or performance overlay
LOD transition	< 16 ms (one frame at 60fps)	Performance profiler — measure re-tessellation time
Frame rate	≥ 60 fps on mid-range hardware	Continuous FPS counter with 1% low tracking

10.3 Automated Testing

43. **Golden image comparison** — render each scene at each LOD level; compare pixel-by-pixel against reference images. Flag if SSIM < 0.99 at max LOD or < 0.95 at min LOD.
44. **Regression tests for manifold parameters** — any change to a ManifoldDescriptor triggers full visual regression suite.
45. **Cross-browser validation** — automated rendering tests on Chrome, Edge, Firefox, and Safari using Playwright + WebGL screenshot capture.

46. **Performance regression** — automated benchmarks track load time, tessellation cost, and frame rate. Alert if any metric degrades by > 10% from baseline.

11. Developer Workflow

11.1 Blender to Manifold

47. **Model in Blender** — use subdivision surfaces for smooth geometry. Design with manifold decomposition in mind: separate hull panels, wings, and structural surfaces as distinct Blender objects.
48. **Identify manifold-eligible surfaces** — surfaces that are smooth, parametric, and can be described by $z = xy$, $z = xy^2$, or compositions thereof. Look for: saddle-like curvature, tubular shapes, swept profiles, repeating lattice elements.
49. **Extract manifold parameters** — using the ButterflyFX Blender addon (in development) or manual parameter fitting:
    - Fit the surface type (xy, xy^2, or custom)
    - Determine domain ranges from the object's bounding box
    - Extract the transform matrix from Blender's object transform
    - Define deformations (taper, twist, bend) by analyzing the surface deviation from the base function
50. **Export remaining geometry** — non-manifold geometry (cockpit, details) exported as standard .glb via Blender's glTF exporter.
51. **Package into hybrid GLB** — the build pipeline combines ManifoldDescriptors + traditional mesh data + textures into a single hybrid .glb file.

11.2 Debugging Tools

ManifoldDebugOverlay

A runtime diagnostic overlay that visualizes:

- **Wireframe mode** — shows tessellation density per patch with color-coded wireframes (green = low density, red = high density)
- **Patch boundaries** — highlights edges between adjacent manifold patches in yellow
- **LOD level indicators** — numeric overlay showing current segment count per patch
- **Budget meter** — bar chart showing triangle budget usage (total, per-category)

Surface Deviation Heatmap

Renders each manifold patch with a color gradient from blue (zero deviation from reference) through green and yellow to red (maximum deviation). Used to validate that manifold parameters accurately reproduce the intended Blender geometry.

Performance Profiler

Tracks per-patch tessellation cost in milliseconds, enabling developers to identify patches that are too expensive and need parameter tuning or lod_bias adjustment.

11.3 Version Control

Asset Type	Storage	Version Control
ManifoldDescriptors	JSON files	Standard Git — fully diffable, mergeable, reviewable in PRs
Traditional mesh data	.glb binary	Git LFS — large binary storage, hash-based deduplication
Textures (source)	.png / .exr	Git LFS

Asset Type	Storage	Version Control
Hybrid GLB (build output)	.glb binary	Not committed — regenerated by CI/CD build pipeline

Diffable Geometry

Because ManifoldDescriptors are JSON, geometry changes show up as meaningful diffs in pull requests: "changed taper rate from 0.35 to 0.42" rather than opaque binary changes. This enables meaningful code review of geometry assets — a significant workflow improvement over binary mesh files.

12. Glossary

Term	Definition
ManifoldDescriptor	A JSON object that fully defines a manifold surface patch — surface type, domain, transform, deformations, tessellation parameters, and adjacency relationships.
Tessellation Density	The number of subdivisions (segments) used when sampling a manifold surface into triangles. Higher density = more triangles = smoother surface.
Saddle Surface ($z = xy$)	A hyperbolic paraboloid — a doubly ruled surface with positive curvature along one diagonal and negative curvature along the other. The primary surface primitive in ButterflyFX.
Asymmetric Surface ($z = xy^2$)	A parametric surface that is linear in x and quadratic in y, producing tapered, intake-like shapes. The secondary surface primitive in ButterflyFX.
Parametric UV	UV texture coordinates derived directly from the manifold function's parameter space (x, y domain), providing

Term	Definition
	distortion-free mapping without manual unwrapping.
Screen-Space Error	The maximum deviation (in pixels) between a tessellated edge and the true manifold curve, as projected onto the screen. Used to drive adaptive LOD.
Hausdorff Distance	The maximum distance between any point on one surface and the nearest point on another surface. Used to measure how closely a tessellated mesh matches its source manifold.
Vertex Quantization	Reducing per-vertex attribute precision (e.g., float32 → int16) to decrease storage size. Controlled by the KHR_mesh_quantization extension.
meshopt	The meshoptimizer library and its associated glTF extension (EXT_meshopt_compression) providing fast-decode geometry compression optimized for GPU consumption.
Draco	Google's geometry compression library (KHR_draco_mesh_compression). Higher compression ratio than meshopt but significantly slower decode speed.
gltfpack	A command-line tool from the meshoptimizer project that optimizes glTF files by quantizing, compressing, simplifying, and reordering mesh data.
KTX2	Khronos Texture 2.0 — a container format for GPU-compressed textures that supports runtime transcoding to platform-native formats (BC, ETC2, ASTC).
Basis Universal	A texture supercompression system by Binomial LLC that encodes textures in a universal intermediate format (ETC1S or UASTC) for transcoding to any GPU texture format.
Hybrid GLB	A GLB file containing both traditional mesh data and ManifoldDescriptor payloads. Compatible with standard viewers; enhanced by ButterflyFX loaders.

Term	Definition
BFXTM_manifold_geometry	The custom glTF extension used by ButterflyFX to embed ManifoldDescriptors within GLB files.
LOD Bias	A per-patch priority value (-1.0 to +1.0) that weights tessellation budget allocation relative to other patches in the scene.
Crack-Free Tessellation	Ensuring that adjacent manifold patches produce identical vertex positions along shared edges, preventing visible gaps (T-junctions) between patches.
C1 Continuity	Matching both position (C0) and first derivative (tangent direction) across patch boundaries, ensuring smooth normal transitions and no visible seams.
BufferGeometry	Three.js class representing GPU-ready geometry via typed arrays for positions, normals, UVs, and indices. The output of the ManifoldGeometryGenerator.
Schwartz Diamond Surface	A triply periodic minimal surface defined implicitly by trigonometric equations. Used as the game board structure in Schwartz Diamond Connect Four.
Instance Array	A set of transform matrices applied to a single geometry definition to render multiple copies efficiently. Critical for lattice structures and asteroid fields.
Octahedral Normal Encoding	A compression technique that maps 3D unit normals to 2D coordinates on an octahedron, reducing normal storage from 12 bytes (3 floats) to 2 bytes (2 int8).
WebGPU Compute Shader	A programmable GPU stage (available via the WebGPU API) that performs general-purpose computation. Used by ButterflyFX for GPU-side manifold tessellation.

13. ButterflyFX Design Suite Index

This document is part of the **ButterflyFX ANPC Design Suite** — a comprehensive set of guideline documents governing the design, development, and optimization of all ButterflyFX game titles.

#	Document ID	Title
1	BFXG-001	General ANPC Guideline Document
2	BFXG-002	Starfighter ANPC Guideline Document
3	BFXG-003	Fast Track ANPC Guideline Document
4	BFXG-004	Schwartz Diamond Connect Four Gameplay Guideline & Manifold Rules Fileset
5	BFXG-005	Starfighter Voice Synthesis & Differentiation Guide
6	**BFXG-006**	**Manifold GLB Optimization Guide (This Document)**

14. References

52. **meshoptimizer / gltfpack** — Arseny Kapoulkine (zeux). Open-source mesh optimization library and GLB compression tool. Repository: zeux/meshoptimizer on GitHub.
53. **Khronos glTF Specification** — glTF 2.0 specification and extensions registry. Khronos Group.
54. **KHR_mesh_quantization** — Khronos extension for storing quantized vertex attributes in glTF. Reduces per-vertex storage by 40–60%.

55. **EXT_meshopt_compression** — Vendor extension providing fast-decode geometry compression. Decode speed ~1 GB/sec with WASM SIMD.
56. **KHR_draco_mesh_compression** — Khronos extension for Draco-compressed geometry in glTF. Google Draco library.
57. **KHR_texture_basisu** — Khronos extension for KTX2 textures with Basis Universal compression in glTF.
58. **Basis Universal** — Binomial LLC. Supercompressed GPU texture transcoding system. LDR/HDR support via ETC1S and UASTC codecs.
59. **Three.js** — ParametricGeometry, BufferGeometry, GLTFLoader, MeshoptDecoder. Three.js r170+.
60. **WebGPU Specification** — W3C GPU for the Web Working Group. Compute shader specification for GPU-side tessellation.
61. **Schwartz Diamond Minimal Surface** — H.A. Schwartz, "Gesammelte Mathematische Abhandlungen" (1890). Triply periodic minimal surface with space group $Fd\bar{3}m$.
62. **Real-Time Rendering Techniques with Hardware Tessellation** — Nießner, M., Loop, C., et al. Adaptive tessellation and displacement mapping techniques for real-time rendering.
63. **ButterflyFX Foundational Surfaces** — $z = xy$ and $z = xy^2$ discovered and applied to game engine geometry by Kenneth Bingham during Blender 3D exploration. These surfaces form the mathematical basis for manifold-native geometry in the ButterflyFX engine.

Author: Kenneth Bingham | Portal: kensgames.com | Engine: ButterflyFX™

© 2026 Kenneth Bingham. All rights reserved. ButterflyFX is a trademark of Kenneth Bingham.